

TriangleFX

From Three Equations to a Valid Triangle

Objective

Create a JavaFX application that takes three equations, finds where the lines meet, checks if they form a valid triangle, and displays the triangle on the screen.

Group 1

Devang Parmar	AU2340217
Dhairya Rupani	AU2340195
Vidhan Nahar	AU2340199
Drumil Bhati	AU2340211

Mentor

Professor Susanta Tewari

Course

CSC360 Computer Graphics and Digital
Image Processing

1. Problem Statement

The Problem

We are given **three equations** representing three lines. Our goal is to find where the lines meet and determine whether they form a valid triangle.

What the system does:

- Takes three equations as input
- Finds the three intersection points
- Checks if the intersection points form a triangle
- Calculates area, perimeter, sides and centroid
- Draws the triangle on a coordinate plane

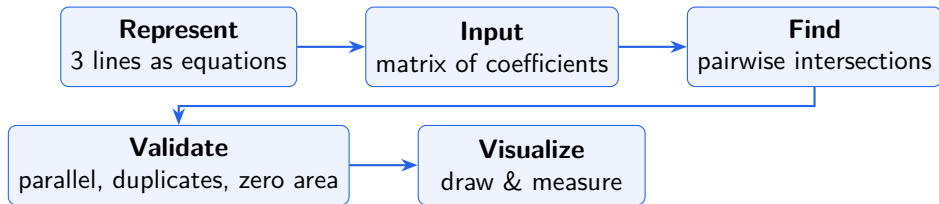
When is it Invalid?

- Two lines are parallel
- Two lines are the same
- All three lines meet at one point
- The three intersection points lie on one line
- Input is missing or invalid

Example

$x + y = 8$, $x - y = 2$, $x = 1$
 $\Rightarrow$ vertices $(5, 3)$, $(1, -1)$, $(1, 7)$

2. Thought Process



Principles

- Represent the three lines using a structured **3×2 matrix + 3 constants**
- Keep mathematical logic **separate from the UI**
- Treat invalid geometry as an expected outcome

How We Built It

- Support direct matrix input and predefined presets
- Solve each pair of lines independently
- Automatically scale the canvas to display the triangle

3. The Mathematics

1. Matrix Formulation ($Ax = b$)

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \\ a_{31} & a_{32} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} \quad \begin{array}{l} \text{Line 1} \\ \text{Line 2} \\ \text{Line 3} \end{array}$$

2. Vertex Intersection (Cramer's Rule)

For lines L_1, L_2 ($D = a_{11}a_{22} - a_{21}a_{12} \neq 0$):

$$x = \frac{b_1a_{22} - b_2a_{12}}{D}, \quad y = \frac{a_{11}b_2 - a_{21}b_1}{D}$$

Gives corners P_{12}, P_{23}, P_{31} ($D = 0 \implies$ parallel).

3. Triangle Properties

Area (Shoelace):

$$\Delta = \frac{1}{2} |x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)|$$

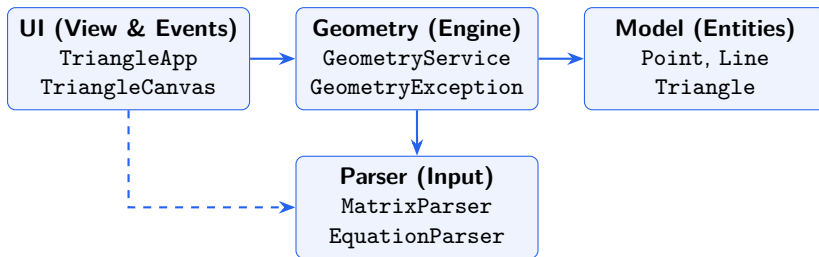
Centroid (Balance): $G = \left(\frac{x_1+x_2+x_3}{3}, \frac{y_1+y_2+y_3}{3} \right)$

Perimeter: $p = a + b + c$ ($d = \sqrt{\Delta x^2 + \Delta y^2}$)

4. Geometric Validation

Valid $\iff D \neq 0$, distinct corners, and $\Delta \geq 10^{-6}$.
(Rejects parallel, coincident, concurrent, or flat lines)

4. The Solution: Layered Architecture



Architecture Principles

- **Decoupled Core:** Math in model/geometry has zero JavaFX UI dependencies.
- **Separation of Concerns:** UI renders; Parser sanitizes; Geometry validates.

Key Engineering Benefits

- **Headless Testing:** All math is 100% testable with JUnit without a display.
- **Clear Failure Feedback:** Custom exceptions explain invalid geometry in the UI.

5. Solution Walk-through: Validation Pipeline

- 1 **Pair check** (3 pairs): identical or parallel $\rightarrow$ error
- 2 **Intersect** each pair (Cramer)
- 3 **Concurrency**: two vertices within tolerance $\rightarrow$ error
- 4 **Area**: if $\Delta < 10^{-6} \rightarrow$ error
- 5 Return the verified Triangle

Every failure carries a human-readable message naming the offending lines.

```
Triangle buildTriangle(Line l1, Line l2, Line l3)
    throws GeometryException {
    // 1. reject identical / parallel pairs
    checkPair(l1, l2, 1, 2);
    checkPair(l2, l3, 2, 3);
    checkPair(l3, l1, 3, 1);

    // 2. the three vertices
    Point p12 = l1.intersectionWith(l2).orElseThrow(..);
    Point p23 = l2.intersectionWith(l3).orElseThrow(..);
    Point p31 = l3.intersectionWith(l1).orElseThrow(..);

    // 3. concurrency: coincident vertices
    if (p12.equalsWithTolerance(p23, TOL) || ..)
        throw new GeometryException("concurrent");

    // 4. degenerate (near-zero area)
    Triangle t = new Triangle(p12, p23, p31, l1, l2, l3);
    if (t.getArea() < MIN_AREA)
        throw new GeometryException("collapsed");
    return t;
}
```

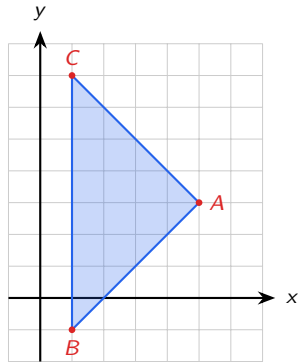
6. Rendering: World-to-Screen Mapping

The canvas fits the triangle to any window size using a single uniform scale, so shapes are never distorted:

$$X_{\text{px}} = \frac{w}{2} + (x - x_c) s$$

$$Y_{\text{px}} = \frac{h}{2} - (y - y_c) s$$

- (x_c, y_c) : centre of the bounding box; s : pixels per unit
- The minus sign flips y (screen y grows downward)
- Grid, axes, lines, fill, and labelled vertices drawn in order



Default example: $A(5, 3)$, $B(1, -1)$, $C(1, 7)$

7. Design Patterns Used

Facade

Class: GeometryService

Method: buildTriangle(...)

The UI calls this one method. It handles equation parsing, line intersections, and triangle validation.

Observer

Class: TriangleCanvas

Observed values: canvas width and height

When the window size changes, JavaFX notifies the canvas. The canvas then calls `redraw()`.

Call from the UI

```
service.buildTriangle(eq1, eq2,  
eq3)
```

Listener in the code

```
widthProperty().addListener(...)  
heightProperty().addListener(...)
```

8. Tests

GeometryServiceTest

- Builds a triangle from equations
- Builds a triangle from a matrix
- Rejects invalid matrix sizes
- Rejects parallel, identical, and concurrent lines
- Checks a right triangle

What these tests check

The geometry service returns the expected points and measurements for valid input, and reports an error for invalid input.

EquationParserTest

- Reads standard equations
- Reads vertical and horizontal lines
- Reads decimal and negative values
- Rejects invalid equations

MatrixParserTest

- Reads both supported matrix formats
- Rejects the wrong number of values

9. Reference Material, Lessons and Next Steps

Reference books (Books/)

- *Core Java, Vol. 1 & 2*: classes, immutability, exceptions, regular expressions
- *Introducing JavaFX 8 Programming* (Weaver): scene graph, Canvas, property listeners
- *Graphics with Java* (2007): 2D drawing and coordinate transformations
- *Digital Image Processing* (Burger & Burge): geometric coordinate mapping

Lessons learned

- Fix the input model first
- Floating point needs explicit tolerances
- Domain exceptions give clear UI messages

Next steps

- Property-based tests
- Export drawing as PNG/SVG
- Generalise to n lines

Thank you — Questions?